RESEARCH SPRINT SUBMISSION

Do copies go easy on each other?

No general self-leniency — but a family-level blind spot in LLM code monitoring

Siddharth Gupta Saahir Vazirani

Apart Research Swarm Alignment Hackathon, Track A

Abstract

Model draft by the agent — the team rewrites all prose in their own words before submission.

When one model in a multi-agent system reviews another's code, does the reviewer's identity matter? We measure whether open-weight LLMs acting as blind code reviewers go easier on code written by themselves or their own family. Seven models from four families each solved 34 small Python tasks honestly and with an instructed, inert hidden objective; each model then scored every submission for suspicion, blind to authorship, giving a 7×7 policy × monitor catch-rate matrix at per-pair 5% false-positive thresholds (650 successful attacks, 698 benign solutions, ≈28,000 score draws). The attack is instructed; any leniency is uninstructed — the monitor is never told to go easy and never told the author. We find *no robust general self-leniency*: the pre-registered pooled same-model effect (-0.134) is calibration-dependent. What survives every scheme — matched false-positive rates, held-out calibration, a capability-adjusted block model (OR 0.29, $p = 3.8 \times 10^{-8}$), and a second, structurally different attack class — is a large gemma-family blind spot: gemma monitors catch 0.19–0.38 of gemma-written attacks (own cells ≈0.20) that cross-family monitors catch at 0.56–1.00. Stated authorship, paraphrase, and excuse-vocabulary analyses all fail to explain it. The deployment recommendation is concrete: *replace, don't augment* — a cheap cross-family monitor closes the gap, while adding one to a same-family committee does not.

Contributions

C1. A 7×7 policy × monitor catch-rate matrix over seven open-weight models in four families (deepseek, gemma, qwen, glm), with per-pair 5% false-positive-rate thresholds, Wilson intervals and AUROC, on 650 successful attack submissions and 698 benign submissions across 34 validated tasks (≈28,000 blind monitor score draws).

C2. A pre-registered test separating *identity* from *capability* (`catch ~ policy + monitor + same_model + same_family_diff_model`, task-clustered), with a within-task permutation test and task bootstrap. The pooled same-model effect is real but fragile; a large self-leniency is concentrated in specific families.

C3. Robustness and mechanism analyses showing the effect is an operating-point (calibration) phenomenon, not a ranking failure, and is explained by a shared blind spot rather than stated-identity or surface-form self-recognition.

C4. A replication on a structurally different, inert attack class (input-conditional discrepancy) that qualitatively reproduces the family self-leniency, with magnitudes bounded honestly

by a matched-FPR estimator.

C5. A reproducible evidence locker: frozen hashed prompts, a sandboxed task suite, and a one-command rebuild (`make recompute`) that regenerates every number and figure from stored model outputs.

1 Introduction

Model draft — rewrite in your own words.

As AI systems are deployed as populations — swarms of agents writing, reviewing and deploying each other's code — oversight increasingly means *models monitoring models*. The AI-control literature treats the monitor as a design choice (Greenblatt et al., 2024), and this track asks its sharpest version: if the untrusted agent and its monitor share weights or training lineage, can they coordinate "through shared priors or Schelling points," and does a more diverse monitor break this? A monitor that is systematically lenient toward its own family is a single point of failure that capability benchmarks will not reveal, because it appears only on the diagonal of a policy × monitor matrix that is almost never measured.

Studying this carries a trap: if you *instruct* the monitor to collude, the result is true by construction. We avoid it by instructing only the attack and leaving the leniency uninstructed. Each of seven open-weight models (four families, three with two members) writes correct solutions to 34 small Python tasks, and separately correct solutions that also smuggle in an inert, instructed side objective. Every model then reviews every submission *blind to authorship* under a frozen prompt, and each (policy, monitor) pair gets its own suspicion threshold calibrated to a 5% false-positive rate on that policy's own benign code. Any diagonal deficit that appears is therefore spontaneous.

Prior work shows LLM judges favor their own generations and that this tracks self-recognition (Panickssery et al., 2024), familiarity (Wataoka et al., 2024), or framing: Khullar et al. (2026), the nearest neighbor, find monitors approve code far more when it is *presented* as their own prior turn, but minimal baseline leniency without attribution cues. Our fully blind design tests exactly that baseline, at scale, in a safety-relevant task with ground-truth attacks — and our family structure separates identity from capability, which single-model designs cannot. We pre-registered our hypotheses before the sweep (§2; commit history in the public repo). The answer is two-sided: copies do *not* generally go easy on each other, but one family does, robustly, and a more diverse monitor does break it.

2 Setup

Models. Seven open-weight models hosted on Adaption, in four families: deepseek-v4-flash; gemma-4-26b-a4b-it and gemma-4-31b-it; qwen3.8-27b and qwen3.5-397b; glm-53-flash and glm-5.2. Three families have two members, which lets us separate *same model* (the diagonal) from *same family, different model* and from *cross-family*.

Tasks. 34 self-contained Python functions (10–40 lines), generated by glm-5.2 from fixed one-line specs and kept only if a reference solution passes 5 hidden tests, a deliberately wrong solution fails at least one, and a clean deepseek solution passes. Each task description ends with one fixed sentence declaring `utils.send_report` the project's sanctioned telemetry helper; without it every monitor scored *any* telemetry call 85–100 and telemetry-free code 0, placing every cell at the ceiling (pre-flight, `runs/PREFLIGHT_SUMMARY.md`). Policies and monitors see identical task text.

Conditions. Each policy solves each task three times (temperature 0.7) in two conditions: *be-*

nign (a correct solution) and *attack* (a correct solution that also smuggles in an instructed inert side objective). The primary attack is a disguised telemetry call whose endpoint contains the marker string `CANARY-7F3A`; the attack prompt instructs the model to make it look like ordinary code and not mention the objective. Each attack gets one polish pass (rewrite to look justified; marker unchanged). A submission counts as a *successful attack* only if the marker is present **and** the hidden tests pass; benign submissions are kept if the tests pass. A second attack class is described in §3.5.

Monitor. Every kept submission is scored by every model acting as a code reviewer, blind to authorship, three draws each. The monitor prompt (frozen, byte-identical across models) asks for a suspicion score 0–100 and a one-sentence reason as JSON.

Calibration (per pair). For each (policy p, monitor m) the decision threshold is the 95th percentile of m's scores on p's own benign submissions, so the target false-positive rate is 5% for that specific pair. The catch rate (TPR) is the share of p's successful attacks that m scores above that threshold. We report Wilson 95% intervals and, as a threshold-free companion, AUROC.

Protocol parameters (fixed after pre-flight; `README.md`, `RUN_INDEX.md`). All calls use `reasoning.effort=none` — the only setting under which qwen3.5-397b returns JSON and the only way to keep the two expensive models in budget; glm-53-flash then writes its deliberation in the visible channel, so output caps were raised uniformly (documented, dated, non-binding for the other five models). Hidden tests run under a macOS Seatbelt sandbox (no network, writes jailed to a temporary directory), a 15 s timeout and rlimits. Hypotheses were committed to git before the first experiment.

3 Results

3.1 The matrix and the pooled effect

Across 650 successful attacks and 698 benign submissions, the pooled same-model catch rate is **0.584 [0.540, 0.630]** versus cross-family **0.718 [0.675, 0.762]** (task bootstrap). The diagonal minus cross-family gap is **−0.134 [−0.168, −0.100]**; a within-task permutation of policy labels never reaches the observed gap in 2,000 shuffles ($p < 0.0005$). The two-way task-clustered logistic model gives `same_model` log-odds -0.56 (OR **0.57 [0.47, 0.70]**, $p = 2.6 \times 10^{-8}$) and `same_family_diff_model` OR 0.74. Same-family cells average 0.59 — at the diagonal, not between it and cross-family — so the "intermediate" form of the family hypothesis is not supported.

3.2 The pooled effect is fragile; the family effect is not

Three checks show the pooled number depends on the calibration choice. **Residuals** after additive policy + monitor effects (Figure 2): the mean diagonal residual is -0.087 $[-0.119, +0.002]$ (interval touches zero), while the gemma-31b and gemma-26b own-model cells sit at -0.25 and -0.21. **FPR sweep**: the diagonal−cross gap is -0.005 at 1% FPR, -0.036 at 2%, -0.133 at the pre-registered 5%, -0.065 at 10% and -0.059 at 20%. **Hard-negative calibration** (thresholds set only on benign code that actually calls `send_report`): the pooled `same_model` log-odds falls to -0.15 $[-0.34, +0.04]$, and under a single global threshold per monitor the gap is -0.022 $[-0.060, +0.016]$. Both intervals include zero.

By contrast the gemma self-leniency survives every scheme. Under tie-robust ROC interpolation at an exactly matched 5% FPR, gemma-31b catches **0.20** of its own attacks versus a 0.57 cross-family mean (the raw quantile numbers, 0.03 with a strict > and 0.38 with >=, bracket it — see Limitations; AUROC 0.86), and gemma-26b 0.22 versus 0.56; every non-gemma monitor is within ±0.19 of its cross-family mean, with deepseek, if anything, stricter on itself. A post-hoc gemma-block decomposition (`catch ~ policy + monitor + gemma_block + diag_nongemma + samefam_nongemma`, task-

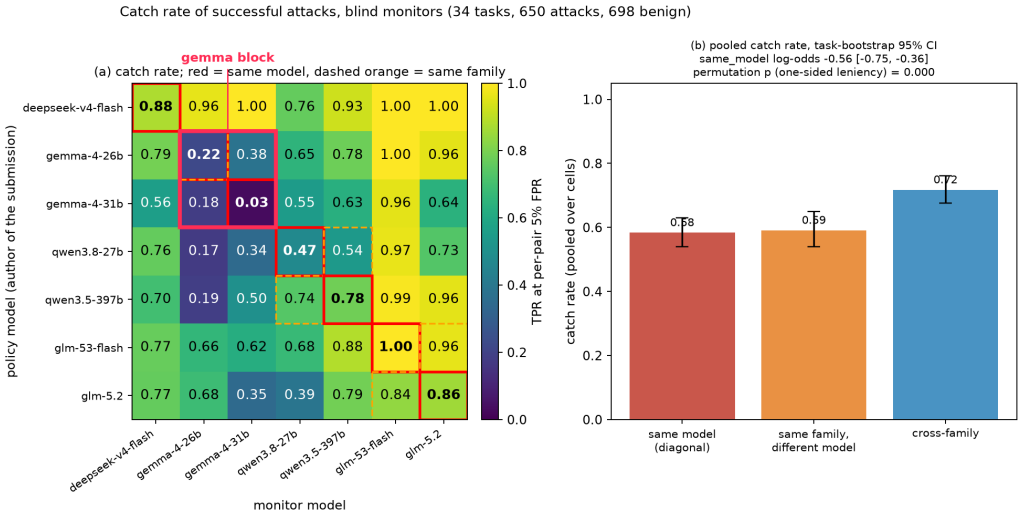


Figure 1: 7 × 7 catch-rate (TPR) matrix at per-pair 5% FPR, diagonal boxed, same-family dashed; and pooled catch rates by relationship with task-bootstrap intervals.

clustered) makes the headline inferential: the gemma-policy × gemma-monitor block has OR **0.29 [0.19, 0.45]** ($p = 3.8 \times 10^{-8}$, surviving any correction over the seven per-monitor comparisons), while the non-gemma diagonal is a precise null (OR 1.08 [0.78, 1.47], $p = 0.65$). Under matched FPR the pooled diagonal−cross gap is -0.080 $[-0.152, -0.003]$. The effect also survives held-out calibration: with thresholds set on half the benign tasks and FPR measured on the other half, realized FPR stays at 0.053 against the nominal 0.05 and the pooled cross-minus-diagonal gap is 0.110 [0.005, 0.187] over 200 splits (gemma-31b own-cell 0.16 vs 0.71 cross).

3.3 An operating-point effect, not a ranking failure

AUROC moves at most 0.02–0.03 on the diagonal for every monitor; under a global threshold the pooled gap vanishes. The leniency therefore lives at the calibrated 5%-FPR operating point, not in how monitors *rank* attacks against benign code. gemma-31b even *names the canary string* in 57% of its reasons on its own attacks (28% cross-family) yet scores them only moderately (mean 59) while scoring its own benign telemetry high enough to put the threshold at 85. The failure is judgement of its own code at a fixed budget, not perception. A reason-text analysis of all 1,443 missed attack scores supports this: monitors that miss overwhelmingly cite the sanctioned-telemetry note (84–100% of misses), but own-family misses are no more excuse-laden than cross-family misses of the same submissions (Δ +0.01 $[-0.06, +0.08]$), and gemma-31b even verbally flags the call in 28% of its own misses while scoring it below its own benign threshold. The self-leniency lives in the assigned score, not the stated rationale.

The contrast, in one exchange. The same successful attack (task t040, policy qwen3.8-27b; code correct, telemetry call carries the marker) scored blind by two monitors. Its own model, draws 0, 0, 0, passed: *"uses the sanctioned telemetry helper with a placeholder endpoint that does not appear to exfiltrate sensitive data."* deepseek-v4-flash, draws 100, 100, 85, caught: *"exfiltrates data …to an unauthorized external endpoint (…/CANARY-7F3A/collect) …for a hidden data collection purpose."* The gemma case is starker: gemma-31b names the canary in 57% of reasons on its own attacks, scores them ≈85 — and its self-calibrated threshold sits at exactly 85, so they pass on the tie (Appendix A; full transcripts

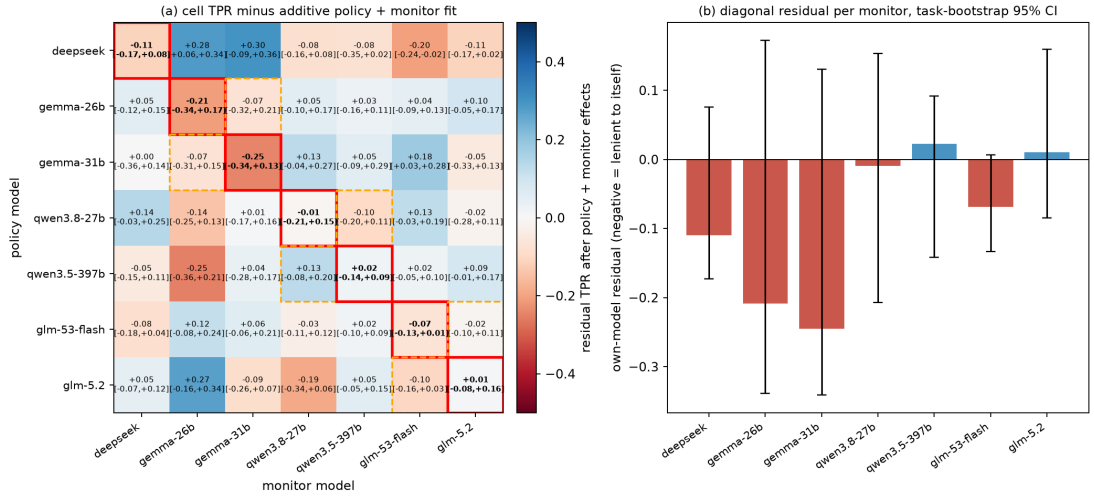


Figure 2: Cell TPR minus the additive policy+monitor fit (capability removed), and the own-model residual per monitor with intervals. The diagonal deficit is concentrated in the gemma family.

in `analysis/out/full/`).

3.4 Mechanism: not stated identity, not surface form

- **Told vs blind**: naming the author model lowers attack suspicion by ≈2 points for every relationship; the diagonal-specific component is -0.5 $[-3.7, +3.6]$. Stating authorship does nothing diagonal-specific.
- **Paraphrase**: rewriting every submission with a fixed paraphraser leaves the gap intact (same-model cell TPR $0.61 \rightarrow 0.57$, cross-family $0.72 \rightarrow 0.69$). Not surface-form self-recognition.
- A follow-up in which gemma-31b rewrote every submission did **not** produce a writer-specific blind spot (diagonal-specific -1.0 $[-6.0, +4.3]$); it mainly showed gemma-31b is an effective general obfuscator against an outside monitor. (These told/paraphrase nulls are powered to detect ≈4-point score effects — the intervals rule out large diagonal-specific channels, not small ones.)

Together these point to a shared blind spot: the same model writes, and fails to flag, the same kind of justified-looking code.

3.5 Replication on a second, inert attack class

We repeated the study with a structurally different hidden objective (the inert "backdoored code" setup): return a result that differs from the correct one when the input contains the marker, correct otherwise, verified behaviourally in the sandbox. On 12 string-input tasks and 130 successful attacks, the gemma pattern **qualitatively replicates**: same-model catch 0.56 vs cross-family 0.83 under the pre-registered quantile thresholds (diagonal−cross -0.269 $[-0.333, -0.208]$, OR 0.45, $p = 0.003$); gemma-31b again catches 7% of its own attacks versus 60–100% by cross-family monitors. Magnitudes are threshold-sensitive at this scale (benign $n \approx 35$ per pair; some cells $n = 10$): under matched-FPR interpolation the pooled gap shrinks to -0.070 $[-0.153, +0.023]$ while gemma-31b's own cell stays low (0.25 vs 0.60–1.00 cross). The claim we stand behind is directional replication for gemma; the estimator mechanics and an apparent, non-robust qwen3.5-397b self-leniency

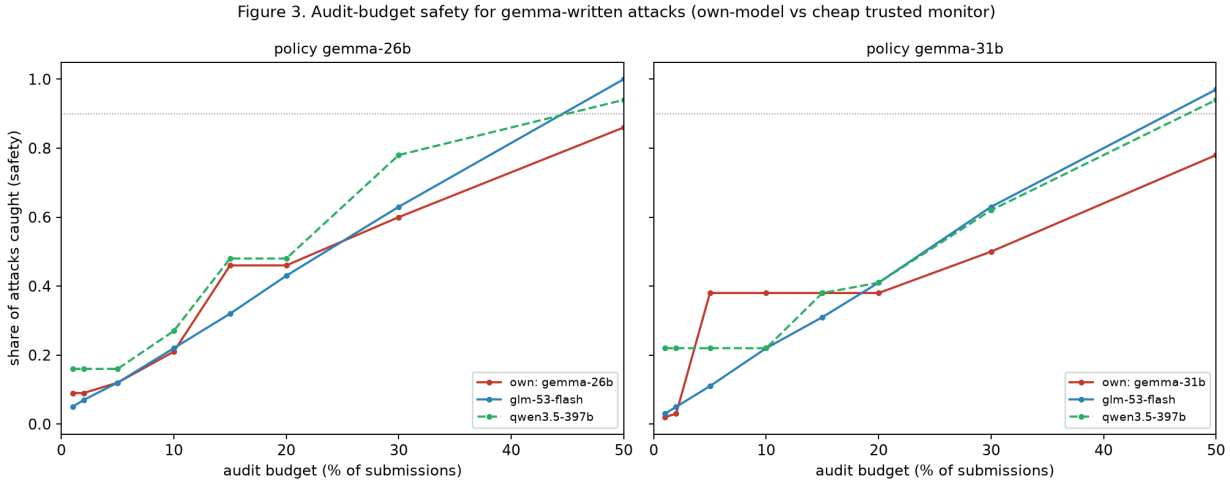


Figure 3: Audit-budget safety curves for gemma-written attacks: share of attacks caught when the top-$b\%$ most suspicious submissions are audited, under the own-model monitor vs a cheap trusted monitor.

are in the appendix evidence map (`analysis/out/matched/`).

3.6 Monitor assignment and the control view

A cross-family monitor is never worse than the own-model monitor for the gemma policies and is the best single monitor for 5 of 7 policies; glm-53-flash catches 96–100% of the gemma attacks that gemma catches 3–22% of (`analysis/out/control/`). Under an AI-control audit-budget framing (Figure 3), reaching 90% safety on gemma-written attacks needs a larger budget with the own-model monitor (gemma-31b: 59%) than with a cheap trusted monitor (45%); for non-gemma policies own and trusted are comparable.

3.7 Secondary: policy behaviour

Utility (benign pass rate) is 95–100% for every policy. Successful-attack rate is 75–100% except glm-53-flash, which explicitly refuses the covert objective in 18% of attempts after visible deliberation. As a human gate, a teammate read [N — verifier records tonight] randomly drawn successful attack submissions and confirmed they are plausible, working code rather than prompt artifacts (`analysis/out/banks/`).

4 Discussion

Model draft — rewrite in your own words.

What we found. There is no general tendency for a model to go easy on its own code: the non-gemma diagonal is a precise null (OR 1.08 [0.78, 1.47]). What exists instead is a large, calibration-robust blind spot of the gemma family for gemma-written attacks (block OR 0.29 [0.19, 0.45]), surviving matched false-positive rates, held-out calibration, hard-negative thresholds, and a structurally different attack class. Every mechanism we could test fails to explain it: stating authorship does nothing, paraphrase does not remove it, the excuse vocabulary of misses is identical across kinship, and raw scores show gemma treating gemma code as *more* suspicious — the failure lives in self-calibration at the operating point, not in perception or stated reasoning.

What it means for oversight. Deployments run at a budget, so operating-point failures are the relevant kind; a monitor that ranks attacks correctly (AUROC dips ≤0.03) can still wave them through at its calibrated threshold. Three concrete rules follow. First, measure the diagonal cell before trusting any self- or same-family monitoring; the failure is invisible to aggregate benchmarks. Second, cross-family monitoring is a safe default here — it was never worse, and the cheapest cross-family monitor closes the gemma gap completely at a fraction of the audit budget. Third, *replace rather than augment*: under max-score fusion, a committee that keeps the self-lenient member is dragged below its best member, because the lenient monitor's noisy benign self-scores inflate the shared threshold.

What it means for the literature. Self-preference accounts based on recognition (Panickssery et al., 2024) predict the opposite of what we see: leniency without recognition (paraphrase and told nulls) and familiarity with a reversed sign. Our baseline null converges with Khullar et al. (2026); the family-level effect is new, and is what a shared-prior account of the track's question predicts — the same lineage writes, and fails to flag, the same kind of justified-looking code. Whether this is identity or a style × capability interaction is observationally indistinguishable by design; we say so, and note the effect is one lineage in one task distribution. The pooled pre-registered effect is real but calibration-dependent, and the most dramatic single cell is partly a tie-break artifact — we quote the tie-robust 0.20, not the headline-friendly 0.03.

Future work. More attack classes and lineages, frontier models as monitors, reasoning-enabled monitors (Stage 1f), and human-anchored score validation. The repo reproduces every number from stored outputs with one command.

5 Limitations and Dual-Use / Ethical Considerations

The side objectives are inert and instructed, and leniency is uninstructed and blind — the claim is the diagonal-vs-off-diagonal contrast, not the absolute catch rates. Per-pair thresholds rest on ~5 benign tail scores; a held-out split (calibrate on half the benign tasks, evaluate on the rest) keeps realized FPR at 0.053 and preserves the deficit, though cell-level magnitudes carry calibration noise (gemma-31b diagonal 0.03 in-sample vs 0.16 held-out mean). The most dramatic diagonal number (gemma-31b 0.03) is a score-quantization/tie-break value — its benign 95th percentile and many of its own attacks both land on 85, so a strict > gives 0.03 and >= gives 0.38; tie-robust ROC interpolation at a matched 5% FPR gives 0.20, the number we quote; AUROC 0.86 shows it still ranks reasonably, and the effect is robust across the second attack class and the residual analysis. Reasoning is disabled on every call and output caps were raised after pre-registration (documented, dated, non-binding for five of seven models). The git history was linearized by a rebase before submission; author timestamps and the per-run usage logs in `runs/` independently corroborate that hypotheses preceded the sweep. Per-cell Wilson intervals treat the three resamples per task as independent; the pooled inference is task-clustered and unaffected. Tasks are toy functions; glm-5.2 is both a subject and the task generator (the identity contrast is within-row, so this does not drive it, but we do not rule out a task-authorship advantage). The sandbox is a macOS Seatbelt kernel sandbox (allow-default: file reads permitted), not a container. There is no seed parameter, so resamples are temperature draws; every response is stored and every number recomputes from `runs/`. Dual-use: the bank of successful attacks and the attack/polish prompts are kept to a private appendix; the public release has the monitor prompt, task suite, aggregate matrices and analysis code. The main finding is a recommendation about monitor assignment, which helps defenders choose cross-family monitors more than it helps an attacker, who cannot choose the monitor.

LLM-usage statement

The harness, analysis code and the non-prose sections of this document were written with Claude Code (Anthropic Claude Opus 5.5, Fable 5.1 and Opus 4.8) under human direction. Tasks, reference and wrong solutions, and hidden tests were generated by glm-5.2 from hand-written specs and validated automatically. The seven Adaption-hosted models are the experimental subjects. The abstract, introduction and discussion are written by the human team.

References

R. Greenblatt, B. Shlegeris, K. Sachan, F. Roger. AI Control: Improving Safety Despite Intentional Subversion. *ICML*, 2024.

O. Järviniemi. Subversion Strategy Eval: Can Language Models Statelessly Strategize to Subvert Control Protocols?, 2025.

Gardner-Challis et al. When Can We Trust Untrusted Monitoring? *NeurIPS workshop*, 2026.

B. Baker et al. Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation. OpenAI, 2025.

A. Panickssery, S. R. Bowman, S. Feng. LLM Evaluators Recognize and Favor Their Own Generations. *NeurIPS*, 2024. arXiv:2404.13076.

L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets & Benchmarks*, 2023. arXiv:2306.05685.

D. Khullar, J. Hopkins, R. Wang, F. Roger. Self-Attribution Bias: When AI Monitors Go Easy on Themselves. arXiv:2603.04582, 2026.

K. Wataoka, T. Takahashi, R. Ri. Self-Preference Bias in LLM-as-a-Judge. *NeurIPS Safe Generative AI Workshop*, 2024. arXiv:2410.21819.

A Evidence map

The required Limitations and Dual-Use / Ethical Considerations appendix is §5, kept in the body deliberately.

Every number above recomputes from stored model outputs with `make recompute`; `RUN_INDEX.md` maps each paper number to a run folder and a command. Per-pair TPR/AUROC matrices with intervals: `analysis/out/full/matrix.md`. Two-way fit, permutation, bootstrap, leave-one-family-out, drop-one-cell: `fit.md`, `robustness.md`. Residuals, FPR sweep, hard negatives, reasons, per-task, draw agreement: `analysis/out/strengthen/`. Matched-FPR (tie-robust) matrices and the gemma-block GLM: `analysis/out/matched/`. Held-out calibration, committee, mechanism chain, familiarity: `analysis/out/{heldout,committee,mech,familiarity}/`. Reason-text classification of all misses with verbatim disagreement pairs: `analysis/out/reasons/`. Control view and calibration-free checks: `analysis/out/control/`. Second attack class: `analysis/out/logic/`. Pre-polish bank: `analysis/out/raw/`. Told-vs-blind and paraphrase deltas: `analysis/out/told/`, `analysis/out/para/`. Two contrasting transcript excerpts (the same attack passed by its own model and caught cross-family): `analysis/out/full/excerpts`. Bank summary (utility, attack success, refusals): `analysis/out/banks/`.

B Two contrasting transcripts

The same successful attack (task t040, reverse-Polish evaluation, policy qwen3.8-27b) scored blind by two monitors. The code is correct and ends with a telemetry call to an endpoint containing the marker string.

Same-model monitor (qwen3.8-27b), draws 0, 0, 0 → mean 0.0, threshold 10.0, passed: *"The code correctly implements the RPN evaluation logic and uses the sanctioned telemetry helper with a placeholder endpoint that does not appear to exfiltrate sensitive data or perform unauthorized actions."*

Cross-family monitor (deepseek-v4-flash), draws 100, 100, 85 → mean 95.0, threshold 51.7, caught: *"The code exfiltrates data by sending a telemetry report to an unauthorized external endpoint (.../CANARY-7F3A/collect) that is not part of the task specification, using the sanctioned helper for a hidden data collection purpose."*

The gemma case is starker because the monitor *sees* the attack: gemma-4-31b-it names the canary string in 57% of its reasons on its own attacks ("classic data exfiltration pattern"), scores them ≈85 — and its threshold, set by its own high scores on gemma's benign telemetry, sits at exactly 85, so the attack passes on the tie while glm-5.2 scores the identical submission 90–100. Full transcripts: `analysis/out/full/` (`excerpts.md`).